

Mapa do Sistema (Inventário)

A estrutura do repositório **fiscalzen-project** é um *monorepo* organizado com Turborepo. Ele contém um backend Fastify (`apps/api`), um frontend Next.js (`apps/web`), pacotes compartilhados para banco de dados, integração com SEFAZ/NFSe, parser de XML e componentes de UI, além de scripts de infra via Docker.

Abaixo está um mapa resumido dos módulos e suas responsabilidades, sempre com evidências de código.

Estrutura de diretórios

Caminho	Papel / Responsabilidade	Evidência
<code>apps/api</code>	API backend Fastify. Define plugins (CORS, rate limit, JWT), autenticação, rotas para empresas, documentos, eventos, dashboard, manifestação, agentes, jobs e NFSe. <code>src/app.ts</code> registra rotas e plugins e expõe <code>/health</code> ¹ .	<code>app.ts</code> mostra o registro de rotas e health check ¹ .
<code>apps/api/src/config</code>	Carrega variáveis de ambiente (usando zod) e configura serviços: banco de dados (Drizzle), Redis, Meilisearch, S3 (MinIO). <code>env.ts</code> define variáveis como <code>DATABASE_URL</code> , <code>REDIS_URL</code> , <code>S3_ENDPOINT</code> , <code>MEILISEARCH_HOST</code> , <code>JWT_SECRET</code> , <code>SEFAZ_AMBIENTE</code> , <code>CERT_ENCRYPTION_KEY</code> ² .	<code>env.ts</code> lista as variáveis obrigatórias ² .
<code>apps/api/src/modules</code>	Cada subpasta implementa um domínio: <code>companies</code> , <code>documents</code> , <code>events</code> , <code>dashboard</code> , <code>manifestacao</code> , <code>agents</code> , <code>jobs</code> , <code>nfse</code> . Cada módulo possui <code>routes.ts</code> (endpoints Fastify) e <code>service.ts</code> (lógica de negócio).	Ex.: <code>companies/routes.ts</code> define rotas CRUD e upload de certificado ³ ; <code>documents/routes.ts</code> define listagem, busca, upload e download ⁴ .

Caminho	Papel / Responsabilidade	Evidência
apps/api/src/jobs	Gerencia filas BullMQ. <code>queues.ts</code> cria filas (<code>sefaz-monitor</code> , <code>xml-processor</code> , <code>search-sync</code> , <code>nfse-monitor</code>) e funções para enfileirar tarefas ⁵ . <code>workers.ts</code> configura workers concorrentes para cada fila ⁶ . Handlers como <code>sefaz-monitor.ts</code> chamam o <code>DistribuiçãoDfe</code> do SEFAZ e enfileiram processamento de XML ⁷ ; <code>xml-processor.ts</code> parseia e persiste documentos ⁸ .	
packages/database	Schema do banco via Drizzle. Define tabelas de <code>tenants</code> , <code>companies</code> , <code>documents</code> , <code>documentEvents</code> , <code>nsu_control</code> , <code>monitor_jobs</code> , <code>audit_logs</code> . Cada tabela inclui chaves estrangeiras para multitenancy. Ex.: <code>documents</code> contém <code>tenant_id</code> , <code>company_id</code> , metadados e índices ⁹ ; <code>nsu-control.ts</code> mantém controle de NSU por empresa/docType ¹⁰ .	Schemas evidenciam multitenancy e controle de NSU ⁹ ¹⁰ .
packages/sefaz-client	Cliente para SEFAZ – implementa Distr. DFe, manifestação do destinatário e manipulação de certificados A1. <code>certificate.ts</code> carrega e valida PFX, calcula dias até expiração e devolve chaves privadas ¹¹ . <code>client.ts</code> cria HTTPS requests assinados com certificado ¹² . <code>services/distdfe-nfe.ts</code> chama a distribuição de NFe, descompacta <code>docZip</code> e retorna XMLs ¹³ . <code>services/manifestacao.ts</code> compõe eventos de ciência/confirmacao/desconhecimento/nao-realizada, assina e envia para SEFAZ ¹⁴ ¹⁵ .	Funções para validar certificado e enviar eventos ¹¹ ¹² ¹³ .
packages/xml-parser	Detecta e extrai campos de XMLs (NFe, CTe, MDFe). <code>detector.ts</code> determina o tipo de documento analisando tags (<code>nfeProc</code> , <code>resNFe</code> , <code>procEventoNFe</code> , <code>mdfeProc</code> etc.) ¹⁶ . Há utilitários para decimais/datas ¹⁷ .	Detecta o tipo de XML ¹⁶ .

Caminho	Papel / Responsabilidade	Evidência
<code>packages/nfse-client</code>	Cliente NFSe (ABRASf e RPA). Contém <code>registry.ts</code> com municípios suportados e tipo de integração (abrasf ou rpa) ¹⁸. <code>factory.ts</code> retorna um cliente <code>AbrasfClient</code> ou <code>BaseNfseScraper</code> conforme município ¹⁹. <code>abrasf/client.ts</code> constrói SOAP, assina XML e consulta NFSe ²⁰. <code>rpa/base-scraper.ts</code> fornece base para scrapers RPA usando Playwright ²¹; <code>municipios/template.ts</code> é modelo para novos scrapers ²². Não há implementações concretas de RPA, apenas o template.	Registro de municípios e cliente ABRASf ¹⁸ ²⁰.
<code>packages/ui</code>	Biblioteca de componentes React (botões, cards, tabelas) compartilhados pelo front-end ²³.	Exporta componentes reutilizáveis ²³.
<code>apps/web</code>	Frontend Next.js (App Router). Tem layout global e páginas dentro de (dashboard): Dashboard, Empresas, Documentos, Manifestação, Upload e possivelmente Jobs/Agents.	
- Dashboard: mostra cards, timeline e integridade usando hooks <code>useDashboardSummary</code>, <code>useIntegrityStatus</code> e <code>useTimeline</code> ²⁴.		
- Empresas: lista empresas usando <code>useCompanies</code> e permite criar/editar, sincronizar SEFAZ e upload de certificado ²⁵.		
- Documentos: exibe documentos filtrados com paginação e busca; usa <code>useDocuments</code> para listar e busca full-text ²⁶.		
- Manifestação: exibe documentos pendentes de ciência/aguardando e histórico; utiliza <code>usePendingManifestations</code>, <code>useAwaitingFinal</code>, etc., e permite enviar eventos ²⁷.		
- Upload: permite arrastar arquivos XML e faz upload via API ²⁸.		

Caminho	Papel / Responsabilidade	Evidência
O <code>api.ts</code> gerencia chamadas à API e token de autenticação; redireciona para login em caso de 401 ²⁹ .	Várias páginas e hooks confirmam as funcionalidades ²⁴ ²⁵ ²⁶ ²⁷ ²⁸ .	
<code>docker/docker-compose.yml</code>	Define stack local com Postgres, Redis, Meilisearch e MinIO com volumes e healthchecks ³⁰ . Inclui serviço <code>minio-init</code> que cria bucket padrão ³¹ .	Compose file mostra configuração da infraestrutura ³⁰ .
<code>.env.example</code>	Lista todas as variáveis de ambiente para config local, como URLs de banco, Redis, S3, Meili, Clerk, JWT, SEFAZ, rate limit e feature flags ³² .	<code>.env.example</code> mostra as variáveis obrigatórias ³² .
<code>apps/api/test</code>	Contém um único teste para as funções de criptografia de certificado ³³ .	Testa round-trip de encrypt/decrypt ³⁴ .

Pontos gerais

- **Multi-tenant:** O schema inclui `tenant_id` e `company_id` em todas as tabelas e as consultas dos serviços usam filtros `tenantId` para isolar dados ³⁵.
- **Filas e workers:** As filas BullMQ `sefaz-monitor`, `xml-processor`, `search-sync` e `nfse-monitor` são criadas e manipuladas. Workers são configurados com concorrência e retry, e o monitor SEFAZ agenda processamentos futuros de NSU ⁵ ⁶.
- **Persistência de XML:** O serviço de documentos usa `@fiscalzen/xml-parser` para detectar e extrair campos, armazena o XML no S3/MinIO via `storage` service, e indexa no Meilisearch ⁸.
- **Manifestação:** O módulo de manifestação consulta o banco para documentos pendentes e usa o cliente SEFAZ para registrar ciência, confirmação, desconhecimento ou operação não realizada; registra eventos no banco ³⁶ ³⁷.
- **NFSe:** O cliente ABRASF está implementado e inclui métodos para consultar notas tomadas/prestadas/por faixa, assinar XML e testar conexão ²⁰. O registro de municípios menciona alguns com integração RPA, mas não há scrapers concretos – apenas um template de base ²², sugerindo que a funcionalidade RPA ainda não foi implementada.
- **Frontend:** O front consome a API via hooks; possui páginas para dashboard, empresas, documentos, manifestação e upload. Não há páginas para NFSe e jobs; a autenticação é feita com Clerk (variáveis definidas), mas no código não aparece integração visível; o `api.ts` utiliza token JWT manual. Portanto a integração Clerk parece parcial.

Checklist de QA

Para cada requisito das áreas A–L foi avaliado o status, com evidências, forma de validação e critério de aceite.

Legenda: OK – implementado e com código consistente; ● Parcial – há código mas faltam peças ou testes; Faltando – não há implementação; Quebrado – implementação presente mas falha evidente; ? Indeterminado – não foi possível validar.

Área / Item	Status	Severidade	Evidência	Como validar	Critério de aceite
A. Infra & Execução					
Docker Compose define Postgres, Redis, Meilisearch, MinIO com volumes e healthchecks		Alta	<code>docker / docker- compose.yml</code> mostra serviços com healthcheck 30	Executar <code>pnpm docker:up</code> deve subir containers; <code>docker ps</code> deve listar serviços; acessar <code>:7700/health</code> deve retornar OK.	Todos os serviços iniciam e ficam saudáveis; bucket <code>fiscalzen- docs</code> criado pelo init.
Scripts <code>pnpm</code> : dev, build, lint, clean		Média	<code>apps/api/ package.json</code> contém scripts <code>dev</code> , <code>build</code> , <code>lint</code> 38 ; <code>root</code> <code>package.json</code> usa <code>turbo</code> para orquestrar tasks 39	Rodar <code>pnpm install</code> e <code>pnpm dev</code> deve iniciar API e Web; <code>pnpm build</code> deve gerar <code>dist</code> ; <code>pnpm lint</code> deve passar sem erros.	Execução dos scripts sem falhas.
Scripts de testes automatizados	●	Alta	Só há um teste de criptografia 34 ; <code>package.json</code> não define script <code>test</code> nas apps 38	Executar <code>pnpm test</code> deve rodar testes; atualmente roda apenas <code>encrypt/ decrypt</code> .	Cobertura de testes unitários e de integração mínima: endpoints, serviços, workers.
Variáveis de ambiente documentadas		Média	<code>.env.example</code> lista todas as variáveis, incluindo Clerk, JWT, SEFAZ, S3, Meili 32	Verificar <code>.env.example</code> e preencher <code>.env.local</code> ; <code>pnpm dev</code> não deve acusar variáveis faltantes.	Todas as variáveis necessárias descritas e validadas por Zod.
Health check da API		Baixa	Endpoint <code>/ health</code> retorna status ok 1	Fazer <code>curl http:// localhost: 3001/health</code> deve retornar JSON com <code>"status":"ok"</code> .	Resposta HTTP 200 com status e versão.

| **B. Segurança & Autenticação** | | | | | Autenticação via Clerk no frontend | ● | Média |
|.env.example define chaves Clerk ⁴⁰; front usa `api.ts` com token manual; não há componentes `SignIn`/`SignedIn` visíveis | Configurar Clerk no Next.js; login deve redirecionar a Clerk; após login, token JWT deve ser salvo e enviado pelo `api.ts` | Usuário consegue logar com Clerk e consumir API; sem login é redirecionado. | | API protegida com JWT | | Alta | Middleware `auth.ts` implementa verificação e roles usando `@fastify/jwt` ⁴¹; rotas aplicam `preHandler` | Enviar requisição sem `Authorization` deve retornar 401; com token válido deve acessar. | Acesso autorizado e escopo por role funcionando. | | Autorização multi-tenant | | Alta | Todos os serviços filtram por `tenantId` nas queries ³⁵; `auth.ts` extrai `tenantId` do token e injeta no request ⁴¹ | Criar duas empresas de tenants diferentes; usuários não devem ver dados de outro tenant. | Consultas de listagem retornam apenas dados do `tenantId` do token. | | Rate limiting | | Média | Plugin de rate limit usa Redis/in-memory; valores configuráveis via env ⁴² | Simular muitas requisições até exceder `RATE_LIMIT_MAX`; resposta deve ser 429. | Respostas 429 quando exceder limites configurados. | | Segredos seguros (certificado, senha) | | Alta | Certificado A1 e senha são criptografados em AES-GCM antes de salvar ⁴³ ³⁵ | Verificar banco: campo `certificate` é binário cifrado; campo `certificatePassword` base64; não há chaves hardcoded. | Credenciais não legíveis em banco; `CERT_ENCRYPTION_KEY` tem 32 bytes. | | Logs de auditoria | ● | Média | Tabela `audit_logs` existe ⁴⁴, mas serviços não registram eventos de upload, manifestação, etc. | Implementar hooks de auditoria; verificar se logs são gravados ao criar/deletar. | Logs persistem usuário, ação e timestamp para eventos sensíveis. |

| **C. Multi-tenant & Modelagem** | | | | | Tabelas com `tenant_id` e `company_id` | | Alta |
Schemas `companies` e `documents` incluem campos `tenant_id` e chaves estrangeiras ⁹; queries filtram por `tenantId` ³⁵. | Inspeccionar banco; as tabelas possuem colunas `tenant_id`, `company_id`. | Todas as tabelas críticas incluem `tenant_id` e rotas exigem esse filtro. | | Filtros obrigatórios | | Alta | Serviços sempre aplicam `tenantId` e `companyId` nas consultas; ex. listagem de documentos filtra `tenantId` e `companyId` opcional ⁴⁵. | Tentar acessar documento de outra empresa via ID; deve retornar 404. | Sem esses filtros não é possível acessar dados de outro tenant. | | Testes de isolamento | | Média | Não há testes de integração que validem isolamento. | Criar testes que tentem acessar dados de outro tenant. | Testes automatizados validam que filtros impedem vazamento de dados. |

| **D. CRUD de Empresas** | | | | | Endpoints CRUD | | Alta | Rotas `/companies` permitem listar, criar, atualizar, deletar; upload de certificado; obter status NSU ³. | Chamar endpoints com JWT e verificar respostas corretas. | Criação retorna 201 com dados; update apenas do próprio tenant; delete remove empresa. | | Validações (Zod) | | Média | Schemas validam campos no `routes.ts`; ex. CNPJ, razão social, etc. | Enviar payloads inválidos deve retornar erro 400 com mensagem amigável. | Erros claros e campos obrigatórios validados. | | Erros padronizados | | Baixa | Utiliza classes de erro (`NotFoundError`, `ValidationError`) e códigos no payload ⁴⁶ ⁴⁷. | Simular erro; resposta possui `error.code`, `error.message`. | Padrão de erros consistente em toda API. |

| **E. Certificado A1** | | | | | Upload e armazenamento seguro | | Alta | Endpoint multipart aceita `.pfx` e senha; serviço valida cert com `@fiscalzen/sefaz-client`, calcula expiração e criptografa conteúdo/senha ³⁵. | Fazer upload de cert válido; banco deve ter campos criptografados; serviço retorna data de validade. | Certificados válidos são aceitos, expiração calculada; falhas retornam erro claro. | | Rotina de expiração | ● | Média | Validação calcula dias até expiração ¹¹, mas não há rotina automática para notificar/renovar. | Implementar cron que verifica `certificateExpiry` e envia alerta. | Sistema avisa quando certificado está próximo de expirar (ex. 30 dias). |

| **F. SEFAZ Monitor (NFe/CTe/MDF-e)** ||||| | Cliente DistDfe implementado | | Alta | `sefaz-client/services/distdfe-nfe.ts` constrói envelope, chama SEFAZ, descompacta `docZip` e retorna XMLs ¹³. | Configurar certificado e ambiente; enfileirar trabalho `sefaz-monitor` e verificar download de XML. | Documentos são baixados e parseados; NSU atualizado. | | Controle de NSU por empresa | | Alta | Tabela `nsu_control` armazena último NSU por empresa/docType ¹⁰; serviço usa e atualiza controle ao monitorar ³⁵. | Sincronizar e ver `nsu_control` incrementando. | NSU persiste e nunca regrede; permite retomada. | | Worker/cron de monitoramento | ● | Alta | Fila `sefaz-monitor` e worker configurados ⁵ ⁶. Não há cron scheduler visível; start manual. | Criar agendador (CronJob) que enfileira monitoramento por empresa; verificar que executa em intervalos configuráveis. | Cada empresa tem job regular que baixa novos documentos. | | Idempotência | | Alta | `xml-processor.ts` verifica duplicidade por hash/chave e não insere repetidos; indexa no search ⁸. | Reenviar mesmo XML; documento não deve duplicar. | Duplicatas são ignoradas ou atualizadas sem criar novos registros. | | Download de XML no frontend | | Média | Página Documentos permite baixar XML via `api.download` e disponibiliza link; search permite visualizar e exportar. | Selecionar documento e clicar em download; deve baixar XML original. | Link funcional e arquivo íntegro. | | Tratamento de falhas/ retry | ● | Média | Workers BullMQ suportam retries automáticos; há uso de `backoff` em `sefaz-monitor` (não encontrado). | Simular timeout; job deve reprocessar com atraso crescente. | Falhas não perdem documentos; logs registram erro. | | Ambientes (homologação/produção) | | Baixa | `env.SEFAZ_AMBIENTE` define 1/2; passado ao cliente ⁴⁸. | Alterar ambiente e verificar que requisições vão ao WS correto. | Configura ambiente via env e repassa ao SEFAZ client. |

| **G. Parser XML & Persistência** ||||| | Detecção de tipo e parser | | Alta | `xml-parser/detector.ts` identifica NFe, CTe, MDFe (proc/res/eventos) ¹⁶; utilitários parseiam campos e datas ¹⁷. | Submeter vários XMLs; parser retorna tipo correto. | Para cada XML oficial, tipo retornado corresponde (NFE, CTE, MDFE) e campos extraídos corretos. | | Persistência idempotente | | Alta | `documents/service.ts` gera hash `sha256Hex` e impede inserção se chave já existir ⁴⁹; armazena XML no MinIO via `storage.ts` ⁵⁰. | Inserir mesmo XML duas vezes; deve atualizar sem duplicar registro. | Documentos únicos por chave de acesso; storage chaveada por hash. | | Armazenamento bruto + metadados | | Média | Campos `xmlKey`, `pdfKey` armazenam chave no S3; metadados gravados no banco ⁹; `storage.ts` faz upload e download ⁵¹. | Verificar S3/MinIO: objeto com mesma chave; metadata associada. | Tanto XML como metadados disponíveis para download e busca. | | Testes com fixtures XML | | Baixa | Não há testes unitários para parser. | Adicionar fixtures NFe/CTe/MDFe e criar testes de parsing. | Testes verificam campos extraídos (valor, data, CNPJ etc.). |

| **H. Manifestação do Destinatário** ||||| | Endpoints e fluxo | | Alta | Serviços `registrarCiencia`, `confirmarOperacao`, `desconhecerOperacao`, `operacaoNaoRealizada` chamam cliente SEFAZ e registram eventos no banco ³⁶ ³⁷. | Chamar endpoints via frontend; observar documento sair de “pendentes” e entrar em “aguardando”/“histórico”. | Eventos enviados com sucesso; status do documento atualizado. | | Automação de ciência | ● | Média | API suporta ciência em lote (`/manifestacao/ciencia/batch`), mas não há automação configurável no cron. | Adicionar job que envia ciência automática para NFe após X dias. | Ciênciapendente é enviada automaticamente e registrada. | | Auditoria | | Baixa | Não há inserção em `audit_logs` ao enviar eventos. | Registrar log (usuário, empresa, evento, hora) em cada ação de manifestação. | Logs persistem todas as manifestações para auditoria. | | Validação de estados | | Baixa | Serviço verifica se documento existe, se certificado está válido e se justificativa é obrigatória em operação não realizada ⁵². | Tentar manifestar documento inexistente ou com certificado expirado deve gerar erro. | Erros corretos retornados. |

| **I. NFS-e (ABRASF + RPA)** | | | | | Cliente ABRASF implementado | | Alta | `AbrasfClient` assina XML, envia e parseia resposta ²⁰ ⁵³ . | Configurar município ABRASF, certificado e testar conexão via `testarConexao`; consultar NFSe de produção/homologação. | Receber retorno de notas tomados/prestadas ou faixa; erros tratados. | | Registro de municípios e factory | | Média | `registry.ts` lista municípios com tipo `abrasf` ou `rpa` ¹⁸; `factory.ts` escolhe cliente conforme tipo ¹⁹ . | Solicitar cliente para município suportado; deve instanciar `AbrasfClient`. | Para municípios suportados, é possível consultar NFSe. | | RPA (scrapers) | | Alta | Apenas existe `BaseNfseScraper` e `template.ts` ²¹ ²²; nenhuma implementação real de município. | RPA está desabilitado por `ENABLE_NFSE_RPA=false`. | Implementar scrapers reais para municípios `rpa` e testes automatizados. | | Frontend de NFSe | | Baixa | Não há página no frontend para NFSe. | Adicionar páginas/rotas para configurar municípios NFSe por empresa e visualizar notas. | Usuário consegue configurar município e consultar NFSe. |

| **J. Busca & Meilisearch** | | | | | Indexação automática | | Alta | `xml-processor.ts` chama `search.index` após persistir documento ⁸; `search-sync` fila atualiza/exclui índices. | Inserir documentos e verificar que são pesquisáveis via `/documents/search`. | Documentos aparecem em buscas imediatamente após processamento. | | Campos filtráveis e pesquisáveis | | Média | `meilisearch.ts` define atributos pesquisáveis e filtráveis ⁵⁴; `search.ts` cria filtros baseados em `tenantId` e `companyId` ⁵⁵. | Realizar buscas por chave, CNPJ, valor; filtrar por empresa. | Campos retornam resultados corretos. | | Fila `search-sync` | | Média | `queues.ts` cria fila `search-sync` ⁵; worker processa reindexação. | Forçar reindexação e verificar queue; search deve refletir mudanças. | Documentos atualizados removidos/apagados também saem do índice. | | Testes de busca | | Baixa | Não há testes de integração para Meilisearch. | Criar testes que inserem documento e buscam por campo. | Testes confirmam indexação e filtros. |

| **K. Dashboard** | | | | | Endpoints summary, timeline, integrity, gaps, recent | | Média | `dashboard/routes.ts` define rotas para resumo, integridade, lacunas e linha do tempo ⁵⁶; frontend chama via hooks `useDashboardSummary`, `useTimeline`, `useIntegrityStatus` ⁵⁷. | Acessar `/dashboard`; cards, gráficos e listagens devem carregar dados reais. | Resumo mostra contagem de documentos, valores, integridade; gaps exibem lacunas de NSU. | | Frontend consome e renderiza | | Média | Página Dashboard mostra cards, gráficos, semáforo e documentos recentes ²⁴. | Usar front e verificar que dados mudam conforme filtros. | Com dados na base, gráficos e semáforo refletem situação. | | Performance (paginação, filtros) | | Baixa | Documentos e manifestacao têm paginação; dashboard timeline solicita últimos 30 dias. Não há caching além do React Query. | Inserir grande volume de documentos e observar carregamento. | Resposta em <3s; paginado para big datasets. |

| **L. Jobs/Filas** | | | | | Filas declaradas | | Média | `queues.ts` declara filas `sefaz-monitor`, `xml-processor`, `search-sync`, `nfse-monitor` ⁵. | Verificar no Redis: todas as filas existem; adicionar jobs retorna OK. | Filas existem e aceitam jobs. | | Workers sobem no `pnpm dev` | | Baixa | `workers.ts` inicia workers com concorrência configurada ⁶; importado em `index.ts`. | Rodar `pnpm dev` deve iniciar workers e processar filas. | Jobs processados conforme esperado; logs exibem progresso. | | Status endpoint e triggers | | Média | `jobs/routes.ts` define endpoints para status de filas, jobs por empresa, sync manual e clean/retry para admin ⁵⁸. | Chamar `/jobs/status` retorna contagem; `/jobs/sync/:companyId` enfileira monitor de SEFAZ; admin pode limpar fila. | Endpoints funcionam; retornos refletem fila atual. | | Observabilidade (dashboard de filas) | | Baixa | Não há UI para visualizar filas ou métricas; depende de logs. | Integrar Bull Board ou UI similar. | Operadores podem visualizar filas e falhas na UI. |

Backlog de Desenvolvimento (Tickets)

Para cada item 🟡 / / , foram criados tickets com prioridade, estimativa e riscos.

Título do ticket	Prioridade	Estimativa	Arquivos impactados	Descrição / Riscos
Adicionar testes unitários e de integração para API e parser	P0	M	apps/api/src/**/*, packages/xml-parser, packages/sefaz-client	Criar suíte de testes usando Vitest ou Node Test. Cobrir endpoints (sucesso/erro), isolamento multi-tenant, parser de XML, SEFAZ monitor, manifestação. Risco: depender de serviços externos; usar mocks.
Implementar agendador para fila sefaz-monitor	P0	M	apps/api/src/jobs/scheduler.ts (novo)	Criar cron que enfileira sincronizações de SEFAZ por empresa com intervalo configurável. Expor configuração via env. Risco: carga na SEFAZ; respeitar rate limit.
Automatizar ciência pendente	P1	M	apps/api/src/modules/manifestacao, jobs	Criar job que envia ciência automaticamente após X dias para NFe sem manifestação. Parâmetro configurável por empresa.
Adicionar logs de auditoria para ações sensíveis	P1	S	apps/api/src/modules/*/service.ts	Em cada serviço sensível (upload de certificado, criação/remoção de empresa, manifestação), inserir registro em audit_logs com usuário, ação e timestamp.
Implementar scrapers RPA para municípios NFSe	P1	L	packages/nfse-client/src/rpa/municipios, apps/api/src/modules/nfse	Desenvolver scrapers Playwright para municípios marcados como rpa (ex. Manaus, Teresina). Usar BaseNfseScraper como base. Riscos: mudanças frequentes no portal, lidar com captcha.

Título do ticket	Prioridade	Estimativa	Arquivos impactados	Descrição / Riscos
Criar página de NFSe no frontend	P2	M	<code>apps/web/app/(dashboard)/nfse</code> , <code>hooks/use-nfse.ts</code>	Interface para cadastrar configurações de NFSe por empresa, acionar sincronização e visualizar notas.
Integrar Clerk no Next.js com proteção de rotas	P2	M	<code>apps/web/app/layout.tsx</code> , <code>lib/providers.tsx</code>	Adicionar componentes do Clerk (<code>SignedIn</code> , <code>SignedOut</code> , <code>SignIn</code>) e Provider; garantir que token JWT é obtido e enviado para API. Risco: conciliar tokens Clerk e Fastify JWT.
Implementar monitor de filas (Bull Board)	P2	S	<code>apps/api</code> , <code>apps/web</code>	Adicionar endpoint que expõe Bull Board ou semelhante e criar página de monitoração no frontend.
Implementar rotinas de expiração de certificado	P2	S	<code>apps/api/src/modules/companies/service.ts</code>	Criar job que verifica certificados próximos da data de expiração e envia notificações por email/webhook.
Adicionar testes de busca Meilisearch	P2	S	<code>apps/api/test</code> , <code>packages/search</code>	Criar testes que inserem documentos e verificam busca por filtros; usar Meili local.
Adicionar testes de NFSe ABRASF	P2	M	<code>packages/nfse-client</code> , <code>apps/api/src/modules/nfse</code>	Criar mocks de serviços ABRASF para testar parser e assinaturas.
Melhorar performance de dashboard	P2	M	<code>apps/api/src/modules/dashboard</code> , <code>apps/web/lib/hooks/use-dashboard.ts</code>	Implementar cache em Redis ou Meilisearch para estatísticas; otimizar consultas SQL com índices e partições.
Adicionar interface de configurações (Configurações gerais)	P3	S	<code>apps/web/app/(dashboard)/configuracoes</code>	Criar página para definir variáveis globais (ex: limites, feature flags) por usuário/empresa.

Título do ticket	Prioridade	Estimativa	Arquivos impactados	Descrição / Riscos
Criar CLI para importar XML em lote	P3	M	<code>apps/api/tools/cli.ts</code>	Desenvolver ferramenta de linha de comando que aceita diretório de XML e processa via API ou diretamente no DB.

Plano de Execução (PRs)

O desenvolvimento deve ser organizado em várias pull requests para facilitar revisão:

1. PR1 – Testes iniciais e infraestrutura de testes (P0)

2. Configurar Vitest ou Node Test para todo o monorepo.
3. Adicionar script `test` em cada `package.json` e no root.
4. Criar testes unitários para utilidades (`encryption`, `errors`) e multi-tenant isolado.
5. Executar `pnpm test` no CI.

6. PR2 – Agendador SEFAZ & Jobs improvements (P0)

7. Criar módulo `scheduler.ts` que agenda sincronizações de SEFAZ via cron (BullMQ repeatable jobs).
8. Adicionar configuração de intervalo via env.
9. Ajustar workers para respeitar `repeatable` jobs e backoff.
10. Atualizar documentação e `.env.example`.

11. PR3 – Manifestação automática & Auditoria (P1)

12. Implementar job de ciência automática para documentos sem manifestação após N dias.
13. Adicionar logs de auditoria em `audit_logs` para upload de certificados, manifestações e exclusões.
14. Criar testes de auditoria.

15. PR4 – NFSe RPA implementation & Frontend (P1)

16. Implementar scrapers para um município RPA (ex. Manaus) usando Playwright; adicionar feature flag para ativar.
17. Criar endpoints de sincronização NFSe no backend (caso ausentes).
18. Adicionar páginas e hooks no frontend para configurar e visualizar NFSe.

19. PR5 – Clerk Auth Integration (P2)

20. Adicionar `ClerkProvider`, `SignedIn` / `SignedOut` e `SignIn` no layout do Next.js.
21. Ajustar `api.ts` para obter JWT a partir de Clerk e armazenar via `setAuthToken`.
22. Criar página de login e logout.

23. PR6 – Monitor de filas & UI (P2)

24. Integrar Bull Board no backend com rota protegida (admin) para visualizar filas.

25. Criar página no frontend mostrando estado das filas e jobs.

26. PR7 – Certificados e expiração (P2)

27. Implementar job diário para checar `certificateExpiry` e notificar usuários via email/webhook.

28. Adicionar testes para essa rotina.

29. PR8 – Melhoria de performance e busca (P2)

30. Adicionar caching de estatísticas de dashboard em Redis.

31. Criar índices adicionais no DB para consultas de timeline e integridade.

32. Implementar testes de performance para Meilisearch.

Cada PR deve incluir atualizações no `.env.example`, scripts, documentação e testes correspondentes.

Aula Rápida

Significado dos Status

- **OK:** A funcionalidade está implementada conforme descrito e o código mostra uma lógica consistente. Ainda assim, deve ser testada em execução real.
- **Parcial:** Existe uma implementação inicial ou parte da funcionalidade, mas faltam etapas (ex.: agendamento, testes, páginas frontend) ou existem feature flags desativadas.
- **Faltando:** Não há código que implemente o requisito ou ele foi prometido mas não realizado (ex.: RPA de NFSe).
- **Quebrado:** A implementação está presente mas claramente incorreta ou gera falhas severas (nenhum caso evidente nesta auditoria).
- **? Indeterminado:** Não foi possível validar com as evidências disponíveis.

Justificativa das Classificações

Para cada item do checklist foi analisado o código fonte e a documentação. Quando o repositório mostrava rotas e serviços completos (ex. CRUD de empresas, parser XML, controle de NSU), o status foi marcado como **OK**. Quando a funcionalidade existia parcialmente (ex. integração Clerk sem UI, jobs sem cron, automação de ciência, NFSe RPA sem scrapers), foi marcado como **Parcial**. Elementos totalmente ausentes, como testes abrangentes ou páginas de NFSe, foram marcados como **Faltando**. Em nenhum ponto foi encontrada lógica obviamente quebrada, por isso **Quebrado** não foi usado.

Uso do Checklist

- O checklist serve como roteiro para QA contínuo. A cada sprint ou PR, o time deve:
1. Revisar itens para garantir que continuam funcionando após mudanças.
 2. Priorizar correção e implementação dos itens **Parcial**/ **Faltando** conforme o backlog, começando pelos de maior severidade/risco.
 3. Adicionar novos itens ao checklist quando surgirem funcionalidades.

4. Utilizar as seções “Como validar” e “Critério de aceite” para criar testes manuais ou automatizados e garantir que a definição de pronto seja cumprida.

Priorização P0/P1/P2

- **P0 (Alta):** Impacta funcionalidades core ou segurança (ex. testes, agendador SEFAZ). Deve ser tratado imediatamente.

- **P1 (Média):** Importante para confiabilidade e experiência do usuário, mas pode aguardar uma sprint (ex. automação de ciência, logs de auditoria, RPA NFSe).

- **P2 (Baixa/Melhorias):** Funcionalidades adicionais ou melhorias de performance/UX (ex. UI de filas, página NFSe, expiração de certificados).

A priorização deve equilibrar valor entregue e risco; funcionalidades críticas para operação (monitoramento SEFAZ, segurança) têm maior prioridade.

1 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/app.ts>

2 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/config/env.ts>

3 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/companies/routes.ts>

4 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/documents/routes.ts>

5 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/jobs/queues.ts>

6 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/jobs/workers.ts>

7 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/jobs/sefaz-monitor.ts>

8 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/jobs/xml-processor.ts>

9 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/database/src/schema/documents.ts>

10 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/database/src/schema/nsu-control.ts>

11 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/sefaz-client/src/certificate.ts>

12 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/sefaz-client/src/client.ts>

13 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/sefaz-client/src/services/distdfe-nfe.ts>

14 15 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/sefaz-client/src/services/manifestacao.ts>

16 raw.githubusercontent.com

<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/xml-parser/src/detector.ts>

17 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/xml-parser/src/utils.ts

18 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/nfse-client/src/registry.ts

19 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/nfse-client/src/factory.ts

20 53 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/nfse-client/src/abrasf/client.ts

21 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/nfse-client/src/rpa/base-scraper.ts

22 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/nfse-client/src/rpa/municipios/template.ts

23 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/ui/src/index.ts

24 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/app/(dashboard)/dashboard/page.tsx

25 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/app/(dashboard)/empresas/page.tsx

26 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/app/(dashboard)/documentos/page.tsx

27 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/app/(dashboard)/manifestacao/page.tsx

28 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/app/(dashboard)/upload/page.tsx

29 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/lib/api.ts

30 31 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/docker/docker-compose.yml

32 40 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/.env.example

33 34 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/test/encryption.test.ts

35 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/companies/service.ts

36 37 48 52 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/manifestacao/service.ts

38 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/package.json

39 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/package.json

41 raw.githubusercontent.com
https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/plugins/auth.ts

- 42 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/plugins/rate-limit.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/plugins/rate-limit.ts>
- 43 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/utis/encryption.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/utis/encryption.ts>
- 44 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/database/src/schema/audit.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/packages/database/src/schema/audit.ts>
- 45 49 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/documents/service.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/documents/service.ts>
- 46 47 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/utis/errors.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/utis/errors.ts>
- 50 51 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/services/storage.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/services/storage.ts>
- 54 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/config/meilisearch.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/config/meilisearch.ts>
- 55 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/services/search.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/services/search.ts>
- 56 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/dashboard/routes.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/dashboard/routes.ts>
- 57 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/lib/hooks/use-dashboard.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/web/lib/hooks/use-dashboard.ts>
- 58 [raw.githubusercontent.com](https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/jobs/routes.ts)
<https://raw.githubusercontent.com/lailtonjunior/fiscalzen-project/main/apps/api/src/modules/jobs/routes.ts>